

Bing Ads Data Warehouse

Technical documentation and setup instructions

1. Overview

This project parses a Bing Ads performance export ([BING_MultiDays.csv](#)) and loads it into a MySQL star schema using Apache Airflow for orchestration. The warehouse supports standard reporting (spend, CTR, conversions by account, campaign, ad, device) and automated insight generation through a single reporting view.

Component	Choice
Orchestration	Apache Airflow 2.9 (LocalExecutor)
Processing	Python 3.9+, pandas
Warehouse	MySQL 8 (database bing_ads_dw)
Driver	PyMySQL with cryptography for MySQL 8 auth
Local environment	Docker Compose
DAG id	bing_ads_performance_etl

The DAG contains wiring only. All logic lives in the [bing_ads_etl](#) package, so the same pipeline can be run from the command line without Airflow.

2. Source data

Property	Value
File	data/raw/BING_MultiDays.csv
Rows	12,497 (plus header)
Columns	37
Date range	2026-01-11 to 2026-01-17 (7 days)
Customers	3
Accounts	9
Campaigns	54 (unique per account + name)
Ad groups	234
Ads	484
Currency / language	USD / English only

The export is fully denormalised: entity attributes (account name, campaign status, ad copy, URLs) repeat on every performance row alongside the metrics.

Handling notes:

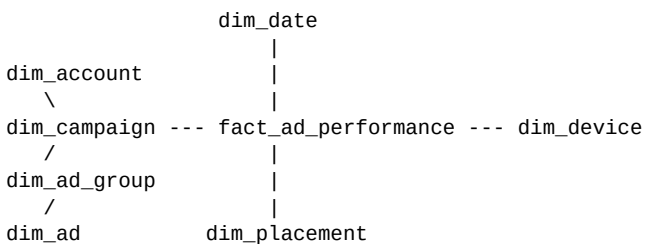
- Ad and ad group IDs are exported in square brackets, for example [\[2709719083\]](#). The transform step strips them.

- Custom Parameters, Final Mobile URL and Final App URL are empty for the whole file. Ad title is empty on most expanded text ads, Final URL is empty on legacy text ads, and one row has no Avg. position. These are loaded as NULL.
- Date format in the source is M/D/YYYY.

Grain check: the combination of date, ad ID, device type, device OS, delivered match type, bid match type, network, top-vs-other and language produces 12,497 distinct values across 12,497 rows. The file is already at its lowest grain, so the pipeline loads it without aggregation.

3. Warehouse schema

Star schema in `bing_ads_dw`, defined in `sql/schema.sql`.



3.1 Dimensions

Table	Surrogate key	Natural key (unique)	Key attributes
dim_date	date_key (YYYYMMDD)	full_date	year, quarter, month, month_name, week_of_year, day, day_name, is_weekend
dim_account	account_key	account_nk (Bing account number)	customer_id, account_name, account_status
dim_campaign	campaign_key	account_nk + campaign_name	campaign_status
dim_ad_group	ad_group_key	ad_group_nk	ad_group_name, ad_group_status, account_nk, campaign_name
dim_ad	ad_key	ad_nk	ad_title, ad_description, ad_type, ad_status, ad_distribution, tracking_template, final_url, final_mobile_url, final_app_url, destination_url, display_url
dim_device	device_key	device_type + device_os	-
dim_placement	placement_key	network + auction_slot + language + delivered_match_type + bid_match_type + currency_code	-

Design points:

- `dim_campaign` is keyed on account plus campaign name, not name alone. Campaign names such as **Brand** repeat across accounts, so a name-only key would merge unrelated campaigns.

- `dim_placement` groups the non-entity auction attributes (network, slot, match types, language, currency). This keeps six low-cardinality columns off the fact table while leaving them filterable.
- Dimensions are Type 1. Reloads update descriptive columns in place via `INSERT ... ON DUPLICATE KEY UPDATE`.

3.2 Fact table

`fact_ad_performance` - one row per date, ad, device and placement.

Column	Type	Notes
<code>fact_key</code>	BIGINT AUTO_INCREMENT	Primary key
<code>date_key</code> , <code>account_key</code> , <code>campaign_key</code> , <code>ad_group_key</code> , <code>ad_key</code> , <code>device_key</code> , <code>placement_key</code>	INT	Foreign keys to dimensions
<code>impressions</code> , <code>clicks</code> , <code>conversions</code> , <code>assists</code>	INT	Additive
<code>spend</code>	DECIMAL(18,4)	Additive
<code>position_weighted_sum</code>	DECIMAL(18,4)	Avg. <code>position</code> x impressions

Constraints and indexes: unique key on (`date_key`, `ad_key`, `device_key`, `placement_key`), foreign keys on all seven dimension keys, and secondary indexes on `date_key`, `account_key` and `campaign_key`.

3.3 Average position

Avg. `position` is a ratio and cannot be summed or averaged across rows, because a position of 1.0 over 5 impressions does not carry the same weight as 2.9 over 95. The fact table stores `position_weighted_sum` instead, so any rollup recovers the correct value:

```
SELECT SUM(position_weighted_sum) / SUM(impressions) AS avg_position
FROM fact_ad_performance;
```

3.4 Reporting view

`vw_ad_performance` joins the full star and exposes:

- Date attributes: `full_date`, `year`, `month`, `day_name`
- Entity labels: `customer_id`, `account_nk`, `account_name`, `campaign_name`, `campaign_status`, `ad_group_name`, `ad_title`, `ad_type`, `display_url`
- Context: `device_type`, `device_os`, `network`, `auction_slot`, `delivered_match_type`, `bid_match_type`, `currency_code`
- Raw metrics: `impressions`, `clicks`, `spend`, `conversions`, `assists`
- Derived metrics: `ctr`, `cpc`, `cost_per_conversion`, `avg_position` (all divide-by-zero guarded)

This is the object analysts and insight agents should query. Example queries are in `sql/insights_queries.sql`.

3.5 Staging table

`stg_fact_ad_performance` holds fact rows with their natural keys during load. Surrogate keys are resolved with one set-based `INSERT ... SELECT` join instead of per-row lookups.

4. Pipeline

`extract -> validate -> transform -> apply_schema -> load`

DAG settings: `schedule="@daily"`, `catchup=False`, 1 retry with a 2 minute delay.

Task	Module	Action	Output
<code>extract</code>	<code>extract.py</code>	Reads the CSV with all columns as strings and verifies the 37 expected columns exist	<code>data/staging/extracted.pkl</code>
<code>validate</code>	<code>validate.py</code>	Applies the data quality rules below	<code>data/staging/validated.pkl</code> , <code>validation_report.json</code>
<code>transform</code>	<code>transform.py</code>	Parses dates, strips bracketed IDs, builds one frame per target table, computes <code>position_weighted_sum</code>	<code>data/staging/transformed/*.pkl</code>
<code>apply_schema</code>	<code>load.py</code>	Executes <code>sql/schema.sql</code> (<code>CREATE ... IF NOT EXISTS, CREATE OR REPLACE VIEW</code>)	Tables and view
<code>load</code>	<code>load.py</code>	Upserts dimensions, replaces facts through the staging table, verifies row count	Populated warehouse

Splitting the work this way means a failure identifies its own cause: a changed file layout fails `extract`, a bad value fails `validate`, an unresolved key fails `load`.

4.1 Validation rules

Fatal (run stops):

- Missing values in required keys: date, customer, account number, campaign name, ad group ID, ad ID, device type, device OS, delivered match type, bid match type, network, top-vs-other, language
- Dates that do not parse as `M/D/YYYY`
- Non-numeric or negative values in impressions, clicks, spend, conversions, assists

Warnings (recorded, run continues):

- Clicks greater than impressions (11 rows in this file)
- Conversions greater than clicks (1 row), which is possible because conversions are attributed back to an earlier click

Empty `Avg. position` is permitted and stored as NULL. All counts are written to `data/staging/validation_report.json`.

4.2 Load behaviour and idempotency

1. Dimensions are upserted by natural key.

2. `fact_ad_performance` and `stg_fact_ad_performance` are cleared.
3. Fact rows are bulk inserted into staging with natural keys.
4. A single `INSERT ... SELECT` joins staging to all seven dimensions to resolve surrogate keys.
5. The loaded fact row count is compared against the input row count and the transaction is rolled back on mismatch.

Step 5 is the safety net: an inner join that misses a dimension row returns fewer rows without raising an error, so the pipeline fails rather than reporting a partial load as success. Reruns produce identical results.

5. Prerequisites

- Docker (Desktop or equivalent) running
- Python 3.9 or newer for the command line route
- Free ports: **3307** for MySQL, **8080** for the Airflow UI

MySQL is published on host port 3307 to avoid conflicting with a local MySQL instance on 3306. Inside the Docker network it still listens on 3306.

6. How to run

Option A - Warehouse and pipeline from the command line

Run these from the project root.

1. Start the database:

```
docker compose up -d mysql
```

2. Confirm it is ready (wait for `healthy` on first start):

```
docker compose ps
```

3. Create a virtual environment and install dependencies:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

On Windows use `.venv\Scripts\activate`.

4. Run the pipeline:

```
PYTHONPATH=src python scripts/run_pipeline.py
```

The script waits and retries if MySQL is still initialising.

5. Stop the database when finished:

```
docker compose stop mysql
```

Option B - Full Airflow stack

1. Build and start all services:

```
docker compose up --build
```

The first build takes several minutes while Airflow dependencies install.

2. Wait for `airflow-webserver` to report healthy, then open `http://localhost:8080`
3. Log in with username `airflow` and password `airflow`.
4. Enable and trigger the DAG `bing_ads_performance_etl`, then watch the five tasks complete in order.
5. Shut down:

```
docker compose down
```

Add `-v` to also delete the MySQL volume and start from an empty database.

The containers mount `./src`, `./dags`, `./data` and `./sql`, so code changes apply without rebuilding. Airflow stores its metadata in a separate `airflow` schema on the same MySQL instance.

Option C - Tests

```
source .venv/bin/activate
pip install -r requirements.txt
pytest -q
```

Two unit tests cover ID bracket stripping and a single row through validate and transform, including the weighted position calculation. Docker is not required.

7. Verifying the load

Expected row counts after a successful run:

Table	Rows
<code>dim_date</code>	7
<code>dim_account</code>	9
<code>dim_campaign</code>	54
<code>dim_ad_group</code>	234
<code>dim_ad</code>	484
<code>dim_device</code>	10
<code>dim_placement</code>	35
<code>fact_ad_performance</code>	12,497

The pipeline prints the same counts on completion:

```
Load complete: {'dim_date': 7, 'dim_account': 9, 'dim_campaign': 54,
'dim_ad_group': 234, 'dim_ad': 484, 'dim_device': 10,
'dim_placement': 35, 'fact_ad_performance': 12497}
```

Query the warehouse:

```
docker compose exec mysql mysql -uads -pads bing_ads_dw -e \
"SELECT account_name, ROUND(SUM(spend),2) AS spend, SUM(clicks) AS clicks,
      SUM(conversions) AS conversions
FROM vw_ad_performance
GROUP BY account_name
ORDER BY spend DESC;"
```

Connection details for any SQL client:

Setting	Value
Host	127.0.0.1
Port	3307
Database	bing_ads_dw
User / password	ads / ads

8. Configuration

Defaults work with the shipped Docker Compose file. Override with environment variables if needed.

Variable	Default	Purpose
SOURCE_CSV	data/raw/BING_MultiDays.csv	Input file path
STAGING_DIR	data/staging	Intermediate files and validation report
SCHEMA_SQL	sql/schema.sql	DDL script
MYSQL_HOST	127.0.0.1	Set to <code>mysql</code> inside Airflow containers
MYSQL_PORT	3307	Host port; 3306 inside Docker
MYSQL_USER	ads	Warehouse user
MYSQL_PASSWORD	ads	Warehouse password
MYSQL_DATABASE	bing_ads_dw	Warehouse schema
PROJECT_ROOT	Inferred from package location	<code>/opt/airflow</code> in Compose

9. Troubleshooting

Symptom	Cause and fix
<code>bind: address already in use on 3306</code>	Another MySQL is running. Compose already publishes 3307, so do not change it back.
<code>'cryptography' package is required</code>	MySQL 8 uses <code>caching_sha2_password</code> . Install from <code>requirements.txt</code> , which pins <code>cryptography</code> .
MySQL was not reachable at <code>127.0.0.1:3307</code>	Container still starting. Check <code>docker compose ps</code> for <code>healthy</code> and rerun.
Fact load lookup failed: expected N rows, inserted M	A natural key did not match a dimension. Inspect <code>data/staging/validation_report.json</code> and the frames in <code>data/staging/transformed/</code> .
Airflow DAG import error	Confirm <code>PYTHONPATH=/opt/airflow/src</code> is set and <code>./src</code> is mounted.

10. Project structure

DOCUMENTATION.md
README.md

this document
short overview

<code>docker-compose.yml</code>	MySQL and Airflow services
<code>Dockerfile</code>	Airflow image with pandas and pymysql
<code>requirements.txt</code>	pinned Python dependencies
<code>dags/bing_ads_etl_dag.py</code>	DAG definition (task wiring only)
<code>src/bing_ads_etl/</code>	
<code>config.py</code>	settings from environment variables
<code>extract.py</code>	CSV read and column contract
<code>validate.py</code>	data quality rules
<code>transform.py</code>	dimension and fact construction
<code>load.py</code>	DDL, dimension upserts, fact load
<code>run.py</code>	runs all steps in sequence
<code>scripts/run_pipeline.py</code>	command line entry point
<code>sql/schema.sql</code>	tables, staging table, reporting view
<code>sql/insights_queries.sql</code>	example analytical queries
<code>sql/mysql-init/</code>	creates the Airflow metadata database
<code>data/raw/BING_MultiDays.csv</code>	source file
<code>tests/</code>	unit tests

11. Current limitations

- Facts are fully replaced each run rather than loaded incrementally by report date.
- Dimensions are Type 1, so status history is not retained.
- No keyword or search query grain, since this export does not contain one.
- Validation runs before load only; there are no post-load warehouse tests.
- Single-machine Airflow with credentials in Compose, not a production deployment.